WORKSHOP

IaC com Terraform

Os fundamentos para criar e gerenciar infraestrutura como código.

AUTOR
Fabio Christiano

PÚBLICO
Time de Engenharia

DURAÇÃO
4h · intervalo de 15 min

FORMATO
Online · com demos ao vivo

ANTES DE COMEÇAR

Combinados

💬 CHAT ABERTO

Pergunte na hora. Respondo ao fim de cada bloco.

🎥 CÂMERA

Se puder, ligue. Ajuda muito quem está apresentando.

☕ INTERVALO

15 minutos, por volta de 1h51 de workshop.

📦 MATERIAL

Slides e código das demos compartilhados no GitHub.

QUEM VAI CONDUZIR O WORKSHOP



Prazer, eu sou o Fabio

Cloud Solution Architect @ Microsoft

www.linkedin.com/in/fabio-christiano

ENQUETE RÁPIDA · RESPONDA NO CHAT

Como vocês criam recursos na Cloud hoje?

A

Console / Portal (clicando)

B

CLI / Scripts

C

IaC do Cloud Provider

- CloudFormation, ARM/Bicep

D

Terraform

AGENDA

O caminho de hoje

IaC e Terraform

Primeira infraestrutura · *demo*

Variáveis e parametrização · *demo*

☕ Intervalo

State: local × remoto · *demo*

Drift · *demo*

Modules e Azure Verified Modules · *demo*

Dependências e repetição · *demo*

Para ir além: funções, Terraform Cloud, OIDC e certificação

Fechamento e perguntas

AO FINAL DO WORKSHOP

O que você leva daqui

- Entender **o que é IaC** e por que ela substitui o trabalho manual
- Conhecer o **ciclo do Terraform**: `init → plan → apply → destroy`
- **Ler** um código Terraform e um plan sem medo
- Entender o **state** e por que ele precisa de cuidado
- Saber **por onde continuar** estudando

Infraestrutura como Código

Por que parar de clicar e começar a versionar.

O PROBLEMA

ClickOps: a infraestrutura que ninguém sabe explicar

? RASTREABILIDADE

Quem criou esse recurso? E por quê?

? CONSISTÊNCIA

Por que **dev** funciona e **prd** não?

? AUDITORIA

Por que o acesso público deste Storage Account foi habilitado?

? RECUPERAÇÃO

Quanto tempo levamos para recriar tudo do zero?

DEFINIÇÃO DE IAC

O que é Infraestrutura como Código?

❌ CONFIGURAÇÃO MANUAL

Rápida, mas cobra o preço depois

- Fácil errar uma configuração: **erro humano**
- Difícil manter o **estado esperado** e a conformidade
- O conhecimento fica na **cabeça de quem fez**

✅ COM IAC

Você escreve um script de configuração para automatizar a criação, atualização ou destruição da infraestrutura de nuvem

- IaC é o **blueprint** da sua infraestrutura
- Fica fácil **compartilhar, versionar e inventariar** a infraestrutura

O que é Terraform e IaC

BENEFÍCIOS DE IAC

Benefícios de IaC na prática

📜 VERSIONADA

Histórico no Git

- Quem mudou, quando e por quê

👀 REVISADA

Pull Request

- A mudança é aprovada antes de chegar na nuvem

🔁 REPETÍVEL

Mesmo código

- dev, hml e prd a partir da mesma base

📖 DOCUMENTAÇÃO VIVA

O código descreve o estado desejado

- Evolui junto com as mudanças revisadas

DUAS FORMAS DE AUTOMATIZAR

Imperativo × Declarativo

IMPERATIVO · *COMO* FAZER

SCRIPT · AZ CLI

```
az group create -n rg-app -l brazilsouth
az storage account create -n stapp001 \
  -g rg-app --sku Standard_LRS
# e se já existir? e se mudou o SKU?
```

DECLARATIVO · *O QUE* DEVE EXISTIR

MAIN.TF · HCL

```
resource "azurerm_storage_account" "app" {
  name                     = "stapp001"
  resource_group_name      = "rg-app"
  account_replication_type = "LRS"
  ...
}
```

➡️ **Imperativo:** você descreve o **passo a passo** para chegar lá.
➡️ **Declarativo:** você descreve o **resultado final** e a ferramenta descobre como chegar lá.

DECLARATIVO+

Terraform é declarativo, com expressões e repetição

DECLARATIVO

YAML · JSON · XML

- Ex.: CloudFormation, ARM Templates
- Expressões e repetição dependem da ferramenta
- Às vezes dá para estender (ex.: CloudFormation Macros)

DECLARATIVO+

HCL · Terraform

- Repetição: `for_each` e `count`
- Valores calculados: `locals`
- Estruturas de dados: maps e coleções

IMPERATIVO

Python · TypeScript · Ruby

- Ex.: Azure SDK, AWS CDK, scripts
- Toda a linguagem de programação à disposição

💡 Continua **declarativo**: descrevemos o resultado, não uma sequência de execução. Veremos `for_each` na reta final.

OPÇÕES DE MERCADO

Ferramentas de IaC: armazenamento em cada uma

ARM TEMPLATES · AZURE

- JSON nativo, verboso · Bicep compila para ARM

```
{ "type": "Microsoft.Storage/storageAccounts",
  "apiVersion": "2023-05-01",
  "name": "stapp001",
  "location": "brazilsouth",
  "kind": "StorageV2",
  "sku": { "name": "Standard_LRS" },
  "properties": {
    "minimumTlsVersion": "TLS1_2",
    "allowBlobPublicAccess": false
  } }
```

BICEP · AZURE

- Sintaxe limpa · compila para ARM · sem state

```
resource sa 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: 'stapp001'
  location: 'brazilsouth'
  kind: 'StorageV2'
  sku: { name: 'Standard_LRS' }
  properties: {
    minimumTlsVersion: 'TLS1_2'
    allowBlobPublicAccess: false
  }
}
```

CLOUDFORMATION · AWS

- YAML ou JSON · equivalente do ARM na AWS

```
Resources:
  AppBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: app-logs-001
      PublicAccessBlockConfiguration:
        BlockPublicAcls: true
        IgnorePublicAcls: true
        BlockPublicPolicy: true
        RestrictPublicBuckets: true
```

TERRAFORM · MULTI-CLOUD

- **HCL para Azure, AWS e GCP** · usa **state**

```
resource "azurerm_storage_account" "app" {
  name                              = "stapp001"
  resource_group_name               = "rg-app"
  location                          = "brazilsouth"
  account_tier                      = "Standard"
  account_replication_type          = "LRS"
  min_tls_version                   = "TLS1_2"
  allow_nested_items_to_be_public   = false
  shared_access_key_enabled         = false
}
```

Trechos de recursos. RG existente no Azure; provider configurado no Terraform. Use nomes únicos para storage e bucket.

CICLO DE VIDA DA INFRAESTRUTURA

Do desenho ao descomissionamento

Fases bem definidas para planejar, construir, entregar, manter e descomissionar a infraestrutura.

FASE 0

Planejar e desenhar

Arquitetura, requisitos, padrões

🛠️ Terraform: escrever os `.tf`

→

FASE 1

Desenvolver e iterar

Construir, testar, ajustar

🛠️ Terraform: `plan` e `apply` em ciclos

→

FASE 2

Ir para produção e manter

Operar, evoluir e, um dia, descomissionar

🛠️ Terraform: `apply` de mudanças, drift e `destroy`

CICLO DE VIDA DA INFRAESTRUTURA

Como IaC melhora o ciclo de vida

CONFIABILIDADE

Mudanças previsíveis

- **Idempotentes**: rodar de novo dá o mesmo resultado
- Consistentes, repetíveis e previsíveis

GERENCIABILIDADE

Mudança via código

- Toda alteração é feita no código
- Revisada e com o mínimo de mudança

CRITICIDADE

Erro na nuvem custa caro

- Storage ou porta aberta por engano → **vazamento de dados**
- Recurso apagado ou alterado na mão → **sistema fora do ar**
- Com IaC, a mudança é **revisada antes** de chegar na nuvem

IDEMPOTÊNCIA

Rodar de novo não pode duplicar a infraestrutura

O código declara **2 VMs**. Faço o deploy, atualizo o código e faço o deploy de novo.

❌ NÃO IDEMPOTENTE

Cria tudo de novo a cada execução

- 1º deploy: sobem 2 VMs
- 2º deploy: sobem **mais 2** VMs

ESPERADO 2 → RESULTADO 4

✅ IDEMPOTENTE

Ajusta o que já existe

- 1º deploy: sobem 2 VMs
- 2º deploy: altera as **mesmas 2** (ou recria, se precisar)

ESPERADO 2 → RESULTADO 2

➡️ O Terraform é idempotente: compara o que você quer com o que existe e aplica **só a diferença**.

CONHECENDO A FERRAMENTA

Terraform

- Criado pela **HashiCorp** em 2014. Hoje parte da **IBM**
- **Binário único** escrito em Go: baixa e roda (Windows, Linux, macOS)
- Linguagem **HCL** (HashiCorp Configuration Language)
- **Agnóstico:** um fluxo de trabalho para qualquer provedor com API

🔗 developer.hashicorp.com/terraform

POR BAIXO DOS PANOS

Quem faz o quê

VOCÊ

Código `.tf`

O que você quer que exista

→

CÉREBRO

Terraform Core

Lê o código, monta o grafo de dependências, compara com o state e calcula o plan

→

PLUGIN

Provider `azurerm`

Traduz cada recurso em chamadas de API

→

AZURE

Resource Manager

Quem de fato cria, altera e apaga

❌ ERRO NO CÓDIGO · TERRAFORM

- Sintaxe, argumento que não existe, referência errada
- Aparece no `validate` ou no `plan`
- Correção: ajustar o `.tf`

❌ ERRO NA NUVEM · API DO AZURE

- Sem permissão (`AuthorizationFailed`), nome já em uso, quota estourada
- Pode aparecer no `plan` ou no `apply`, repassado pelo provider
- Correção: **RBAC/quotas no Azure** ou **nome/configuração no código**

Providers

GRAFO DE DEPENDÊNCIAS

Você descreve, o Terraform ordena

MAIN.TF · TRECHO · FORA DE ORDEM, DE PROPÓSITO

```hcl
resource "azurerm_subnet" "app" {
  virtual_network_name = azurerm_virtual_network.vnet.name
  ...
}
resource "azurerm_network_security_group" "nsg" {
  resource_group_name = azurerm_resource_group.rg.name
  ...
}
resource "azurerm_virtual_network" "vnet" {
  resource_group_name = azurerm_resource_group.rg.name
  ...
}
resource "azurerm_resource_group" "rg" { ... }
```

Referências e dependências

DEPENDÊNCIAS DE CRIAÇÃO



- A ordem vem das **referências**, não da ordem no arquivo
- O que não depende entre si roda em **paralelo**
- Quem monta esse grafo é o **Terraform Core**
- NSG criado **sem associação** neste exemplo

O ECOSSISTEMA

Providers: os plugins que falam com cada serviço

OFFICIAL

Mantidos pela HashiCorp

- `azurerm`, `aws`, `google`, `random`

PARTNER

Mantidos pelo fabricante

- `datadog`, `cloudflare`

COMMUNITY

Mantidos pela comunidade

- Qualquer pessoa pode publicar

PROVIDERS.TF

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}

provider "azurerm" {
  features {}
}
```

registry.terraform.io

> Terraform **não** é "escreve uma vez, roda em qualquer nuvem". O que é igual é o **fluxo de trabalho**.

SABER O LIMITE

Cada camada, sua ferramenta

✅ É PAPEL DO TERRAFORM

- Recursos de nuvem: Azure (rede, storage, AKS, Key Vault), AWS, GCP
- GitHub: repos, teams, branch protection
- Entra ID, Datadog, Cloudflare...

❌ NÃO É PAPEL DELE

- Configurar o sistema operacional
- Instalar pacotes dentro da VM
- Fazer deploy da aplicação

CAMADA	FERRAMENTA
Provisionar a infraestrutura	**Terraform / Bicep / CloudFormation**
Configurar o que roda dentro	**Ansible / Puppet / cloud-init**
Entregar a aplicação	**Pipeline · Helm · Argo**
Tarefa pontual / consulta	**CLI (az, aws)**

Primeira infraestrutura

Do arquivo vazio ao recurso no Azure.

SETUP

O que você precisa na máquina

TERRAFORM CLI

O binário

- `winget install Hashicorp.Terraform`

AZURE CLI

Para autenticar

- `winget install Microsoft.AzureCLI`

VS CODE + EXTENSÃO

HashiCorp Terraform

- Autocomplete, validação e formatação

GIT

Código mora em repositório

- Infraestrutura também

🔗 Instalação para Windows, Linux e macOS: developer.hashicorp.com/terraform/install

A LINGUAGEM

Anatomia de um bloco HCL

MAIN.TF

```hcl
resource "azurerm_resource_group" "rg" {
  name     = "rg-workshop-demo1-a7k2"
  location = "brazilsouth"

  tags = {
    env = "dev"
  }
}
```

- **Tipo do bloco:** `resource`
- **Tipo do recurso:** `azurerm_resource_group` (provider + recurso)
- **Nome local:** `rg`, como *você* chama esse recurso no código
- **Argumentos:** `chave = valor`

Comentários: #, // ou / */ · diferencia maiúsculas de minúsculas*

Sintaxe do HCL

ORGANIZAÇÃO

Um projeto Terraform é uma pasta

ARQUIVO / PASTA	PARA QUE SERVE	ORIGEM / GIT
`main.tf`, `rg.tf`, `storage.tf`	Resources organizados por assunto; nomes por convenção	Você escreve · Git ✅
`providers.tf`	Versões do Terraform/providers e configuração dos providers	Você escreve · Git ✅
`variables.tf`	Declara entradas: tipo, default e validação	Você escreve · Git ✅
`outputs.tf`	Declara valores que o projeto expõe como saída	Você escreve · Git ✅
`terraform.tfvars`	Atribui valores às entradas; carregado automaticamente	Você preenche · Git ✅ se não houver segredos
`.gitignore`	Exclui cache, states, plans salvos e arquivos com segredos	Você escreve · Git ✅
`.terraform/`	Binários dos providers e módulos baixados	`init` gera · Git ❌
`.terraform.lock.hcl`	Registra versões escolhidas e checksums dos providers	`init` gera · Git ✅
`terraform.tfstate`	State local: mapa entre código e recursos gerenciados	Terraform gera · Git ❌
`terraform.exe` (Windows)	Executável da CLI; roda `init`, `plan`, `apply`...	Instalado no `PATH`, normalmente fora do projeto

💡 `main.tf` **não é obrigatório nem executa primeiro.** Todos os `.tf` da pasta formam uma configuração; as dependências definem a ordem.

⚠️ A leitura **não é recursiva**: `storage.tf` na mesma pasta entra; `storage/main.tf` em uma subpasta só entra se for chamado como módulo.

FIXE VERSÕES

O bloco `terraform {}`

PROVIDERS.TF

```
terraform {
  required_version = "⩾ 1.9"

  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "⇝ 4.0"
    }
  }
}
```

CONSTRAINT	SIGNIFICA
`= 4.10.0`	Somente 4.10.0
`⩾ 4.0`	≥ 4.0.0, sem limite superior ⚠️ Inclui 5.x e posteriores
`⇝ 4.0`	≥ 4.0.0 e < 5.0.0 ✅
`⇝ 4.10.0`	≥ 4.10.0 e < 4.11.0

*A constraint define a **faixa permitida**; `.terraform.lock.hcl` registra a **versão escolhida do provider**.*

🔑 Fixe versões.

Version constraints · Dependency lock file

CONECTANDO NO AZURE

Configurando o provider

PROVIDERS.TF

```hcl
provider "azurerm" {
  features {}
  # subscription_id → via ARM_SUBSCRIPTION_ID
}
```

BASH

```bash
az login
export ARM_SUBSCRIPTION_ID="$(
  az account show --query id -o tsv | tr -d '\r'
)"
```

⚠️ No `azurerm` **4.x**, o `subscription_id` é **obrigatório**.

🔑 Na sua máquina: **Azure CLI**.
No pipeline: **OIDC** (S80).

LIGANDO RECURSOS

Um recurso usa o outro

MAIN.TF

```
resource "azurerm_resource_group" "rg" {
  name     = "rg-workshop-demo1-a7k2"
  location = "brazilsouth"
}

resource "azurerm_virtual_network" "vnet" {
  name                = "vnet-workshop-demo1-a7k2"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  address_space       = ["10.10.0.0/16"]
}
```

- 🔗 Referência: `<TIPO>.<NOME>.<ATRIBUTO>`
- ➡️ **Dependência implícita:** o Terraform cria o RG **antes** da VNet
- 💡 **Argumentos** você define; **atributos** o Azure devolve (`id` , `address_space`)

DO CÓDIGO À INFRAESTRUTURA

O ciclo de comandos do Terraform

1 **init** — Prepara o diretório → 2 **fmt** — Padroniza o código → 3 **validate** — Verifica a configuração → 4 **plan** — Mostra o que vai mudar → 5 **apply** — Executa as mudanças → 6 **destroy** — Remove os recursos

⚠️ `destroy` é uma **saída opcional** para remover os recursos, não o próximo passo obrigatório.

🔑 Leia o plan antes de aplicar.

Workflow do Terraform

PASSO 1 · PREPARAR

terraform init

BASH

```
terraform init

Initializing the backend...
Initializing provider plugins...

Terraform has been successfully initialized!
```

- Baixa **providers** e **módulos**
- Inicializa ou atualiza o **backend**
- Cria `.terraform/`
- Cria ou atualiza `.terraform.lock.hcl`
- Repita ao mudar provider, módulo ou backend

> 💡 É seguro repetir. O `init` prepara o projeto; **não cria recursos no Azure**.

terraform init

PASSO 2 · PADRONIZAR

terraform fmt

DESENVOLVIMENTO

```
terraform fmt

# Pipeline: verifica sem editar
terraform fmt -check
terraform fmt -check -recursive
```

- Ajusta espaços, alinhamento e estilo oficial do HCL
- `fmt` **corrige** os arquivos
- `-check` apenas **verifica**
- Não muda o significado da configuração
- Evita discussão de estilo no Pull Request

> 💡 Código formatado não significa código válido. O próximo passo é verificar a configuração.

terraform fmt

PASSO 3 · VERIFICAR

terraform validate

MAIN.TF · ERRO PROPOSITAL

```
resource "azurerm_resource_group" "rg" {
  name     = "rg-workshop-demo1-a7k2"
  locaton = "brazilsouth"
}
```

TERRAFORM VALIDATE

```
Error: Unsupported argument

An argument named "locaton"
is not expected here.
```

- Verifica **sintaxe e consistência interna**
- Valida **tipos, referências e argumentos**
- Usa o schema dos providers baixado pelo `init`; normalmente requer `terraform init`
- Não cria recursos nem verifica permissões no Azure
- Não substitui o `plan`

> 🔑 **Código válido ainda pode produzir uma mudança perigosa.**

terraform validate

PASSO 4 · REVISAR

terraform plan

TERRAFORM PLAN

```
# azurerm_resource_group.rg will be created
+ resource "azurerm_resource_group" "rg" {
    + id       = (known after apply)
    + location = "brazilsouth"
    + name     = "rg-workshop-demo1-a7k2"
  }

Plan: 3 to add, 0 to change, 0 to destroy.
```

- Compara **configuração, state e infraestrutura real**
- Mostra a mudança proposta **sem executá-la**
- **+** criar · **~** alterar no lugar · **-** destruir
- **-/+ destruir e recriar** ⚠️ (pode perder dados!)
- Pode salvar o plano revisado:
  `terraform plan -out=tfplan`

> 🔑 Sempre leia a **última linha** primeiro e procure qualquer destruição inesperada.

terraform plan

PASSO 5 · EXECUTAR

terraform apply

BASH

```bash
# Calcula um novo plan e confirma
terraform apply

# Executa exatamente o plano salvo
terraform apply tfplan
```

- Sem arquivo: calcula um novo plan e pede confirmação
- Com `tfplan`: executa **exatamente o plano salvo e revisado**
- Cria, altera ou remove recursos **de verdade**
- Atualiza o **state** com o resultado
- Sem diferenças entre código e infraestrutura: `No changes`

> 🔑 Em automação, aplique o plano que foi revisado: `terraform apply tfplan`.

terraform apply

SAÍDA OPCIONAL · REMOVER

terraform destroy

BASH

```
# Revise a remoção primeiro
terraform plan -destroy

# Calcula e executa a remoção
terraform destroy
```

- Usa o **state** para localizar os recursos gerenciados; `plan -destroy` permite revisar a remoção antes
- Útil para laboratórios e ambientes temporários
- Não é obrigatório depois de todo `apply`
- Exige confirmação antes de executar

> ⚠️ Destruir infraestrutura pode significar **perder dados**. Trate o destroy com o mesmo cuidado de um apply.

terraform destroy

▶ DEMO 1

Do zero ao Azure

01 `az login` e a subscription certa

02 Criar `main.tf` · `init → fmt → validate`

03 `plan -out=tfplan → apply tfplan` → conferir no Portal

04 `apply` de novo: **idempotência**

05 Editar o mesmo arquivo: tags (`~`) · nome novo (`-/+`, só plan)

06 `destroy`

📁 demos/demo1-rg-vnet · um main.tf ao vivo · state local na mesma pasta

RECAP

O que acabamos de ver

- ✅ Projeto Terraform = **pasta com** `.tf` + providers com **versão fixada**
- ✅ Ciclo: **init** → **fmt** → **validate** → **plan** → **apply** → **destroy**
- ✅ **Leia o plan.** Cuidado com o `-/+`

Variáveis e parametrização

Um código, vários ambientes.

O PROBLEMA

Copiar e colar não escala

❌ ANTES

Uma pasta por ambiente

- `dev/main.tf` e `prd/main.tf`, 95% iguais
- Corrigiu em um, esqueceu no outro

✅ DEPOIS

Um código + valores por ambiente

```
main.tf  +  dev.tfvars  →  ambiente dev
main.tf  +  prd.tfvars  →  ambiente prd
```

ENTRADAS

Declarando uma variável

VARIABLES.TF

```
variable "env" {
  type        = string
  description = "Ambiente: dev ou prd"
  default     = "dev"

  validation {
    condition     = contains(["dev", "prd"], var.env)
    error_message = "env deve ser dev ou prd."
  }
}
```

Input variables

- Uso: `var.env`
- **Tipos:** `string` · `number` · `bool` · `list` · `map` · `object`
- **validation** barra valores errados **antes** de ir ao Azure
- 🔒 `sensitive = true` esconde o valor da saída do terminal

ATRIBUINDO VALORES

De onde vêm os valores

DEV.TFVARS

```
env      = "dev"
location = "brazilsouth"
tags     = { owner = "time-plataforma" }
```

TERMINAL

```
terraform plan -var-file="dev.tfvars"
```

PRECEDÊNCIA · A ÚLTIMA VENCE

1. `default` na declaração
2. Variável de ambiente `TF_VAR_env`
3. `terraform.tfvars` (automático)
4. `*.auto.tfvars` (automático)
5. `-var-file` **e** `-var` na linha de comando

VALORES CALCULADOS

Locals: evitar repetição

● LOCALS.TF + MAIN.TF

```
locals {
  name_prefix = "${var.project}-${var.env}"
  common_tags = merge(var.tags, {
    env        = var.env
    managed_by = "terraform"
  })
}

resource "azurerm_resource_group" "rg" {
  name     = "rg-${local.name_prefix}"
  location = var.location
  tags     = local.common_tags
}
```

Locals

- Campeão no Azure: **naming e tags**
- `managed_by = "terraform"` avisa quem abrir o Portal

> 💡 **variable** = entrada de fora · **local** = cálculo de dentro

SAÍDAS

Outputs: o que o código devolve

OUTPUTS.TF

```
output "vnet_name" {
  description = "Nome da rede virtual"
  value       = azurerm_virtual_network.vnet.name
}

output "vnet_id" {
  value = azurerm_virtual_network.vnet.id
}
```

TERMINAL

```
$ terraform output -raw vnet_name
vnet-workshop-dev-demo2-a7k2
```

Outputs

- Ver o resultado sem abrir o Portal
- Alimentar o **pipeline** (`-raw`)
- Expor dados de um **módulo**

USANDO O QUE JÁ EXISTE

Data sources: ler sem gerenciar

Um **data source é uma consulta ao provider**: obtém informações para usar na configuração.

DATA.TF

```
data "azurerm_virtual_network" "hub" {
  name                = "vnet-hub"
  resource_group_name = "rg-connectivity"
}

output "hub_vnet_id" {
  value = data.azurerm_virtual_network.hub.id
}
```

RESOURCE	DATA
Cria, altera e destrói	Só lê

- **Busca:** consulta a VNet pelo nome e pelo Resource Group.
- **Retorna:** atributos como `id`, `location` e `address_space`.
- **Usa:** `data.azurerm_virtual_network.hub.id` pode alimentar um resource, local ou output.
- **Não gerencia:** não cria, altera ou exclui a VNet; a consulta precisa de acesso de leitura no Azure.

> 💡 **Resource gerencia; data consulta.** Use o que a plataforma já criou, sem assumir seu ciclo de vida.

Data sources

▶ DEMO 2

Um código, dois ambientes

01 Valores fixos viraram `variables.tf`

02 `locals` para naming e tags

03 `plan` com `dev.tfvars` × `prd.tfvars`

04 Validação barrando um valor errado

05 `apply` em dev · `outputs` e um data source

demos/demo2-variables

☕ INTERVALO

15:00

Iniciar | Reiniciar 15:00

Pronto para iniciar.

Voltamos às --:--

🤔 PARA PENSAR

Onde o Terraform guardou o que ele criou?
E se duas pessoas rodarem `apply` ao mesmo tempo?

State

A memória do Terraform e o arquivo mais importante do projeto.

A MEMÓRIA DO TERRAFORM

`terraform.tfstate`: o mapa entre código e mundo real

TERRAFORM.TFSTATE · EXEMPLO ILUSTRATIVO

```
"type": "azurerm_storage_account",
"name": "sa",
"attributes": {
  "id": "/subscriptions/<sub-id>/resourceGroups/...",
  "primary_access_key": "Xk9f...==",    ⚠️
  ...
}
```

- 🗺️ Liga `azurerm_storage_account.sa` ao recurso real no Azure
- 📋 Guarda os **atributos** de tudo o que foi criado
- 🔗 Guarda as **dependências** entre recursos

> ⚠️ O state guarda **segredos em texto puro**.

🔑 **O state é o mapa. Proteja o mapa.**

State

COMO O PLAN É CALCULADO

Três versões da verdade

CÓDIGO

O que eu quero

Arquivos `.tf` e `.tfvars`

⇄

STATE

O que o Terraform acha que existe

A memória da última execução

⇄

AZURE

O que realmente existe

Consultado via API

① REFRESH

Azure → State: "como está cada recurso que eu conheço?"

② DIFF

Código × State → as mudanças do plan

POR QUE LOCAL NÃO SERVE PARA TIME

State no notebook = risco

💥 PERDA

Formatou a máquina?

- O Terraform "esqueceu" a infraestrutura

🔓 SEGREDOS

Chaves em texto puro

- No disco de cada pessoa (ou no Git)

👥 COLABORAÇÃO

Cada um com o seu

- States divergentes

🔒 LOCK APENAS LOCAL

Não coordena states em máquinas diferentes

❌ **Nunca** commitar o `.tfstate` no Git.

A SOLUÇÃO

Backend azurerm: state no Azure Storage

BACKEND.TF

```
terraform {
  backend "azurerm" {
    resource_group_name  = "rg-tfstate"
    storage_account_name = "sttfstate<sufixo>"
    container_name       = "tfstate"
    key                  = "workshop/dev.tfstate"
    use_azuread_auth     = true
  }
}
```

Backend azurerm

✅ Centralizado ✅ Criptografado ✅ Lock nativo ✅ Versionável

🔑 `use_azuread_auth`: acesso via **Entra ID (RBAC)**, sem access key.

O bloco `backend` **não aceita variáveis**.

UM DE CADA VEZ

State locking

TERRAFORM PLAN

```
Error: Error acquiring the state lock

Lock Info:
  ID:        8c1f2e4a-...
  Path:      tfstate/workshop/dev.tfstate
  Operation: OperationTypeApply
  Who:       maria@NOTE-123
  Created:   2026-10-20 14:32:10 UTC
```

[State locking]

- 🔒 No Azure Storage, o lock é um **lease** no blob, automático
- A mensagem diz **quem**, **o quê** e **desde quando**
- ⚠️ `terraform force-unlock <ID>` : **só** se tiver certeza de que ninguém está rodando

PROTEJA O MAPA

Checklist do storage de state

✅ VERSIONAMENTO DE BLOB

Volta para uma versão anterior

✅ SOFT DELETE

Recupera se alguém apagar

✅ RBAC MÍNIMO

Só pipeline e admins (`Storage Blob Data Contributor`)

✅ REDE RESTRITA

Firewall ou private endpoint com acesso público desabilitado

✅ RESOURCE LOCK

`CanNotDelete` no storage

✅ SEPARADO

RG ou subscription de gerenciamento

ORGANIZAÇÃO

Um state por ambiente e por projeto

CONTAINER TFSTATE/

```
├── network/dev.tfstate
├── network/prd.tfstate
├── app-pagamentos/dev.tfstate
└── app-pagamentos/prd.tfstate
```

Comandos terraform state

- 💥 **Raio de impacto menor:** um erro afeta só aquele state
- 🔍 `terraform state list`: lista os recursos gerenciados
- 🔍 `terraform state show <endereço>`: mostra os atributos

> ❌ **Nunca** edite o `.tfstate` na mão.

▶ DEMO 3

Dev e prd, states separados

01 Mesmo código · dois backends locais e dois `.tfvars`

02 Inspecionar dev → `plan`: **No changes**

03 Selecionar prd com `init -reconfigure`

04 Aplicar prd → dev e prd coexistem

05 Voltar ao backend de dev → **No changes**

📁 demos/demo3-remote-state · um state local por ambiente

Drift

Quando o Azure e o código param de concordar.

DEFINIÇÃO

Drift = a realidade mudou fora do código

1 **Terraform cria** → 2 **State registra** → 3 **Alguém muda no Portal** → 4 `plan` **detecta**

Causas comuns: Hotfix na madrugada | "É só uma tag" | Outro script/automação | Azure Policy

O código diz uma coisa. O Azure, outra. **Quem está certo?**

Manage resource drift

A DECISÃO

Detectou drift. E agora?

A · O CÓDIGO VENCE

A mudança foi indevida

- `terraform apply` **reverte** para o que está no código

B · A MUDANÇA ERA LEGÍTIMA

Leve a mudança para o código

- PR atualizando o código → `plan` volta a "No changes"

❌ **C · Deixar como está:** um próximo `apply` pode desfazer a mudança se o plan não for revisado.

💡 `terraform apply -refresh-only` atualiza o **state**, não o código.

▶ DEMO 4 ⭐

Alguém mexeu no Portal...

01 Na VNet: mudar `owner` e incluir `centro_custo`

02 `terraform plan` propõe reverter as duas tags

03 Decidir: **o Portal está correto**

04 `apply -refresh-only` → atualizar só o state

05 `state show` confirma · o código ainda precisa acompanhar

📁 demos/demo4-drift · alteração + inclusão · refresh-only

PREVENÇÃO

Portal para consultar. Código para mudar.

🔐 RBAC

Pessoas com **Reader** em prd

- Quem escreve é o pipeline

📜 AZURE POLICY

Bloqueia o proibido

- Ex.: storage com acesso anônimo

🚀 PIPELINE

Único caminho para mudanças

- PR → plan → apply

⏰ DETECÇÃO

`plan` agendado

- Alerta quando há diferença

RECUPERAÇÃO DO STATE

Perdi o state. E agora?

Os recursos continuam no Azure. O Terraform perdeu o mapa que os liga ao código.

1 · PARE E CONFIRA

Suspenda applies e pipelines

- Confira backend, container e chave do blob.

2 · RESTAURE UMA CÓPIA VÁLIDA

Backup local ou versão anterior do blob

- Proteção precisa estar habilitada e a cópia disponível.

3 · SEM BACKUP? RECONSTRUA

`import` associa recursos existentes ao código/state

- `aztfexport` pode ajudar no Azure, mas não recupera o state original.

4 · REVISE ANTES DE RETOMAR

Execute `plan`

- Reconcilie diferenças entre código, state recuperado e Azure antes de qualquer `apply`.

> ⚠️ **Não rode `apply` com state vazio para "recuperar".** Restaurar é o plano A; importar é o plano B.

Import de recursos existentes (plano B)

Modules

Escreva uma vez, use em todo lugar.

O PROBLEMA

10 projetos, 10 redes, 10 jeitos diferentes

- ❌ Bloco copiado e colado entre projetos
- ❌ Um esqueceu a tag de gestão, outro errou o naming
- ❌ Corrigir significa caçar em todos os repositórios
- ✅ **Um módulo** com o padrão da empresa, e todo mundo consome

CONCEITO

Módulo = pasta de `.tf` reutilizável

INPUTS · VARIABLES

`name`
`address_space`
`tags`

→

MÓDULO NETWORK

azurerm_virtual_network

+ espaço de endereços
+ managed_by obrigatório

→

OUTPUTS

`id`
`name`

- 🌳 **Root module:** a pasta onde você roda `terraform` · 🌿 **Child module:** o módulo que o root chama
- 🔒 Nas expressões, o root referencia os **outputs**, não os recursos internos

Modules

NA PRÁTICA

Chamando um módulo

MAIN.TF

```
module "shared" {
  source = "./modules/network"

  name                = "vnet-shared-dev"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  address_space       = ["10.20.0.0/16"]
  tags                = local.common_tags
}

output "shared_vnet_id" {
  value = module.shared.id
}
```

- 🔗 Saída do módulo: `module.<NOME>.<OUTPUT>`
- O consumidor não precisa saber de como impor a tag de gestão

> ⚠️ Adicionou ou mudou um módulo? **`terraform init`** de novo.

FONTES E VERSÕES

Onde moram os módulos

FONTE	EXEMPLO DE SOURCE
Pasta local	`./modules/network`
Terraform Registry	`Azure/avm-res-network-virtualnetwork/azurerm` + `version`
Git	`git::https://github.com/org/tf-modules.git//network?ref=v1.2.0`
Registry privado	`app.terraform.io/<org>/network/azurerm`

📌 **Fixe a versão exata:** `version = "1.2.0"` no Registry · `?ref=v1.2.0` no Git

💡 Módulo compartilhado entre times → **repositório próprio, com tags de versão**

Sources e versões de módulos

NÃO REINVENTE

Azure Verified Modules: módulos oficiais da Microsoft

🧱 RESOURCE MODULES

Um recurso, completo

- `Azure/avm-res-<serviço>-<recurso>/azurerm`

🏗️ PATTERN MODULES

Cenários compostos

- `Azure/avm-ptn-<padrão>/azurerm`

- ✅ Interfaces padronizadas, conforme suporte do recurso: **diagnostic settings · RBAC · private endpoints · locks · managed identity**
- ✅ Alinhados ao **Well-Architected Framework**, testados e mantidos pela Microsoft
- ✅ Disponíveis para **Terraform e Bicep** · 🔗 `aka.ms/avm`

NA PRÁTICA

Usar AVM ou escrever o seu?

AVM DIRETO

Quando o padrão Microsoft atende

MÓDULO "FINO" EM CIMA DO AVM

Quando a empresa tem regras extras

- Naming, tags obrigatórias, SKUs permitidos

❌ **Evite:** módulo próprio do zero só para "embrulhar" um único recurso sem agregar regra nenhuma.

▶ DEMO 5

Do bloco ao módulo

01 Resource Group e VNet: um módulo por recurso

02 Módulo `subnet` chamado duas vezes na mesma VNet

03 Outputs conectam RG → VNet → subnets

04 `plan`: **5 to add** · random + 4 recursos Azure

05 Revisar e executar `apply`

📁 demos/demo5-modules · código único · módulos locais

Dependências e repetição

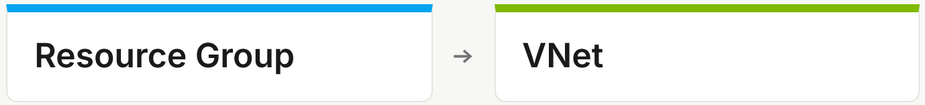
Quem vem primeiro? Como criar vários sem copiar e colar?

DEPENDÊNCIA IMPLÍCITA

A referência já diz quem vem primeiro

MAIN.TF · TRECHO

```
resource "azurerm_virtual_network" "vnet" {
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  # Demais argumentos omitidos
}
```



- A referência ao RG cria uma **dependência implícita**.
- Não precisa adicionar `depends_on` para repetir essa relação.

💡 A ordem não vem do nome do arquivo nem da posição do bloco.

DEPENDÊNCIA EXPLÍCITA

Quando a referência não conta tudo

VM · TRECHO ILUSTRATIVO

```hcl
resource "azurerm_linux_virtual_machine" "app" {
  # Demais argumentos omitidos
  identity {
    type         = "UserAssigned"
    identity_ids = [azurerm_user_assigned_identity.app.id]
  }
  depends_on = [azurerm_role_assignment.kv_reader]
}
```

- **Implícita:** VM referencia a identidade gerenciada.
- **Explícita:** a VM espera a permissão da identidade via Azure RBAC, pois a aplicação lê segredos do Key Vault ao iniciar.

⚠️ `depends_on` ordena operações; **não garante a propagação do RBAC**.

💡 **Referências primeiro. depends_on só para dependências que o código não expressa.**

depends_on

DECLARATIVO+ NA PRÁTICA

count: quantos? for_each: quais?

Os dois repetem um bloco `resource`. A diferença é como identificam cada instância.

COUNT · QUANTIDADE

Quero 3 Resource Groups

- `count = 3` → três instâncias.
- Identificadas no state por número: `[0]`, `[1]`, `[2]`.

FOR_EACH · CHAVES

Quero RGs para dev, hml e prd

- Um map com as chaves `dev`, `hml` e `prd` → uma instância por entrada.
- Identificadas no state pela chave: `["dev"]`, `["hml"]`, `["prd"]`.

💡 **count usa posição; for_each usa chave.** Os dois continuam declarativos.

`count` · `for_each`

FOR_EACH

A chave identifica; o valor configura

LOCALS.TF + MAIN.TF · TRECHO

```
locals {
  environments = {
    dev = "brazilsouth"
    hml = "eastus"
  }
}

resource "azurerm_resource_group" "env" {
  for_each = local.environments

  name     = "rg-workshop-foreach-${each.key}"
  location = each.value
}
```

- `each.key`: `dev` ou `hml` → nome do RG.
- `each.value`: `brazilsouth` em dev ou `eastus` em hml → região do RG.
- Endereços: `azurerm_resource_group.env["dev"]` e `["hml"]`.

FOR_EACH · ANTES E DEPOIS

Entrou prd. Dev e hml continuam os mesmos.

Adicionamos uma chave ao map, sem mudar as entradas existentes.

ANTES: DEV + HML	DEPOIS: DEV + HML + PRD	PLAN
`env["dev"]`	`env["dev"]`	Sem alteração
`env["hml"]`	`env["hml"]`	Sem alteração
—	`env["prd"]`	**Criar**

Endereço completo, por exemplo: `azurerm_resource_group.env["dev"]`.

✅ **Plan: 1 to add, 0 to change, 0 to destroy.**

⚠️ Renomear uma chave não é só mudar uma etiqueta: pode planejar remoção e criação. **Leia o plan antes de aplicar.**

▶ DEMO 6

Duas subnets, um map

01 Mesmo cenário da Demo 5: RG, VNet e duas subnets

02 `locals`: `app` e `data` → prefixos das subnets

03 Um bloco `module.subnet` com `for_each`

04 `plan` + `apply`: **5 to add** · endereços por chave

📁 demos/demo6-for-each · módulos reutilizados · state próprio

⚠️ Duas subnets na mesma VNet. **for_each não isola ambientes nem permissões.**

Para ir além

Funções do HCL, Terraform Cloud, OIDC e seu próximo passo.

MAIS EXPRESSIVIDADE NO HCL

Built-In Functions: transformar e combinar valores

Funções nativas recebem valores e devolvem um resultado para usar na configuração.

LOCALS.TF · EXEMPLO

```
locals {
  projeto = lower("WORKSHOP")
  nome_rg = format("rg-%s-dev", local.projeto)
  tags = merge(
    { projeto = local.projeto },
    { ambiente = "dev" }
  )
}
```

- **lower:** `"WORKSHOP"` → `"workshop"`.
- **format:** monta o nome `"rg-workshop-dev"`.
- **merge:** combina tags de projeto e ambiente em um map.

Catálogo de Built-In Functions

UMA PLATAFORMA GERENCIADA

Terraform Cloud — hoje HCP Terraform

Plataforma da HashiCorp para executar e acompanhar o Terraform.

STATE REMOTO

Armazenamento e histórico de versões

EXECUÇÃO REMOTA

`plan` e `apply` com histórico de runs

INTEGRAÇÃO COM GIT

Mudanças no repositório podem iniciar runs

💡 **É opcional:** Terraform CLI + backend Azure Storage continuam sendo um caminho válido.

🔗 Conheça o HCP Terraform

AUTENTICAÇÃO DO PIPELINE

OIDC: GitHub → Azure, sem client secret

OIDC (OpenID Connect): o workflow prova sua identidade usando um token, não uma senha guardada.

GITHUB ACTIONS
Token OIDC
Identifica a execução

→

MICROSOFT ENTRA ID
Confiança federada
Valida o token e emite um access token

→

TERRAFORM → AZURE
Token temporário
Chama APIs do Azure com permissões RBAC

- ✅ Nenhum **client secret** guardado para autenticar.
- 🔒 Restrinja a confiança ao **repo, branch ou environment** e conceda **RBAC mínimo**.

🔗 Guia oficial: OIDC GitHub → Azure

UMA TRILHA OPCIONAL

HashiCorp Certified: Terraform Associate (004)

- Valida fundamentos de Terraform; **não é uma certificação específica de Azure**.
- Inclui IaC, providers, configuração, workflow, state, modules e HCP Terraform.
- Caminho de estudo: **praticar → revisar objetivos oficiais → preparar-se para o exame**.

> 💡 Este workshop é a base, **não cobre todo o exame** e não substitui a preparação.

🔗 Certificação e guia oficial

RECAP

Missão cumprida?

- ✅ Entender **IaC, HCL e providers** e executar o **ciclo do Terraform**
- ✅ Parametrizar com **variables, locals e outputs** e consultar **data sources**
- ✅ Entender **state local/remoto**, locking, proteção e recuperação e **identificar e corrigir drift**
- ✅ Reutilizar **módulos**, entender **dependências** e repetição com `count` e `for_each`
- ✅ Conhecer **funções nativas, HCP Terraform, OIDC e certificação** como caminhos de continuidade

MENSAGEM FINAL

Portal para consultar.
Código para mudar.

OBRIGADO